

Peripherals

Basic properties:

Peripherals are IO devices using which the processor can communicate with the external world.

Function of IO devices

Act as data transducers i.e., to convert information from one physical representation to another.

Do not alter the information content or meaning of the data on which they act.

Since data is transferred and processed within a computer system in the form of digital electrical signals, input devices transform other form of information to digital electrical signals and output devices transform digital electrical signals to other form of information.

Many of such devices use electromechanical technology and hence slower than CPU and main memory.

Bus: The various processor level components like CPU, IOP, Memory, IO or peripheral devices of a computer system are interconnected by buses. The system components communicate with each other through bus.

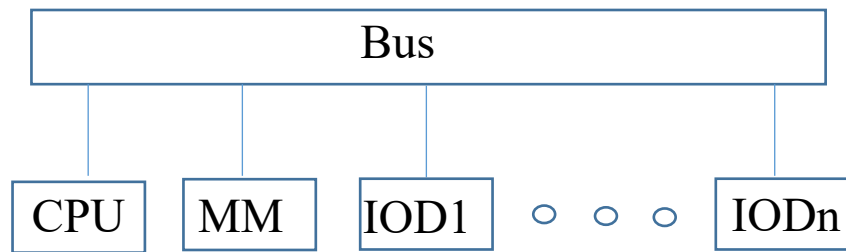


Fig.1: Single bus system

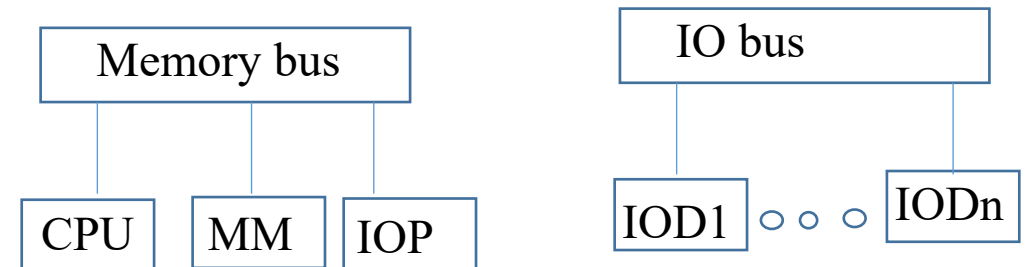


Fig.2: Double bus system

Bus is shared among CPU, main memory (MM) and **n** number of IO devices in single bus system

In double bus system, memory bus is used for communication among CPU, MM and IO processor (IOP) whereas IO bus is used for communication between IOP and **n** number of IO devices

Communication between MM and CPU is straightforward but the communication between IOP and IO devices is a source of difficulty due to the wide variety of operating characteristics (such as speed, some devices are electromechanical type etc.) of IO devices.

Bus control: As multiple devices are connected to the same bus so it is required to control the time of transferring information through the bus and to select one device from multiple devices that can gain access to the bus.

Devices are categorized as bus master and bus slave.

Bus master

Main bus control unit. CPU and IOP are example of bus master.

Supervises the use of bus by other devices.

Drives most bus lines including certain lines that can't be driven by any slave units, for example, CPU can drive all data, address and most control lines of a system bus.

Bus slave

Responds command issued by bus master. Memory and IO interface circuit are example of bus slave.

Each IO device has an IO interface circuit.

MM and IO interface circuit can drive data line and control line but can't drive address line (i.e. MM and IO interface circuit cannot generate address, address is always generated by master device).

Function of IO interface circuit (Fig.5):

Contains an address decoder to decode the address which is placed by CPU on the address bus to see whether this address is the address of its associated IO device. If the address on the address bus and the address of the IO device are not identical the IO interface circuit ignores the address on the address bus. Otherwise it starts communication between its associated IO device and CPU.

Control circuit in the IO interface has bidirectional connection with the control bus for sending status signals and receiving control signals (for example, IOR for IO read in case of input device, IOW for IO write in case of output device).

IO interface circuit generates the appropriate control signals that are required by the bus control scheme. For example, the status signal determines whether the communication between CPU and IO device is successful or not, occurrence of error if any etc.

Data and status register in the IO interface circuit has bidirectional connection with the data bus to store status information, to receive data (in case of output device for write operation) and to send data (in case of input device for read operation).

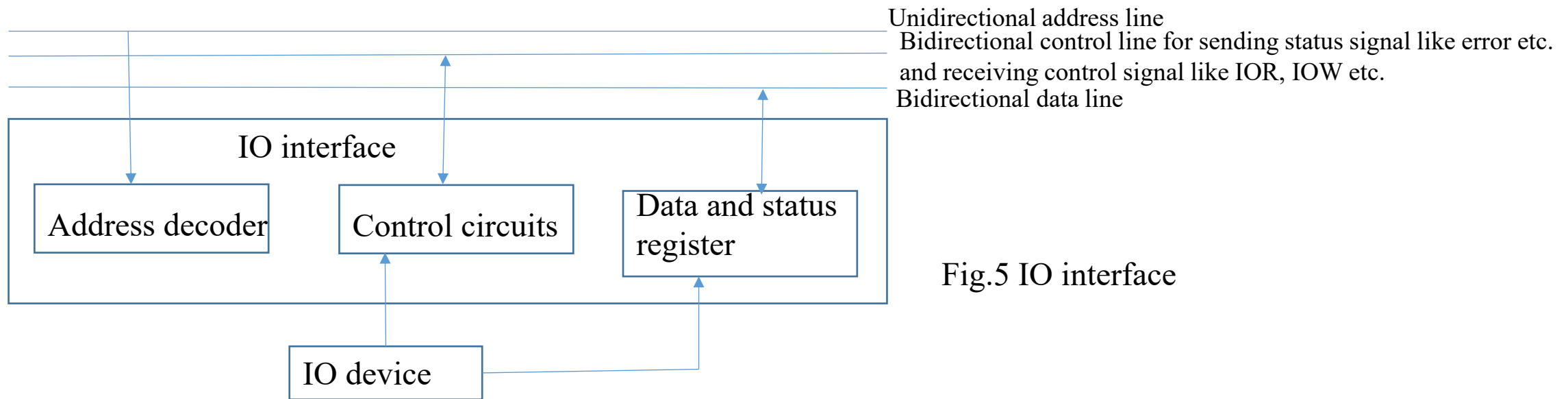


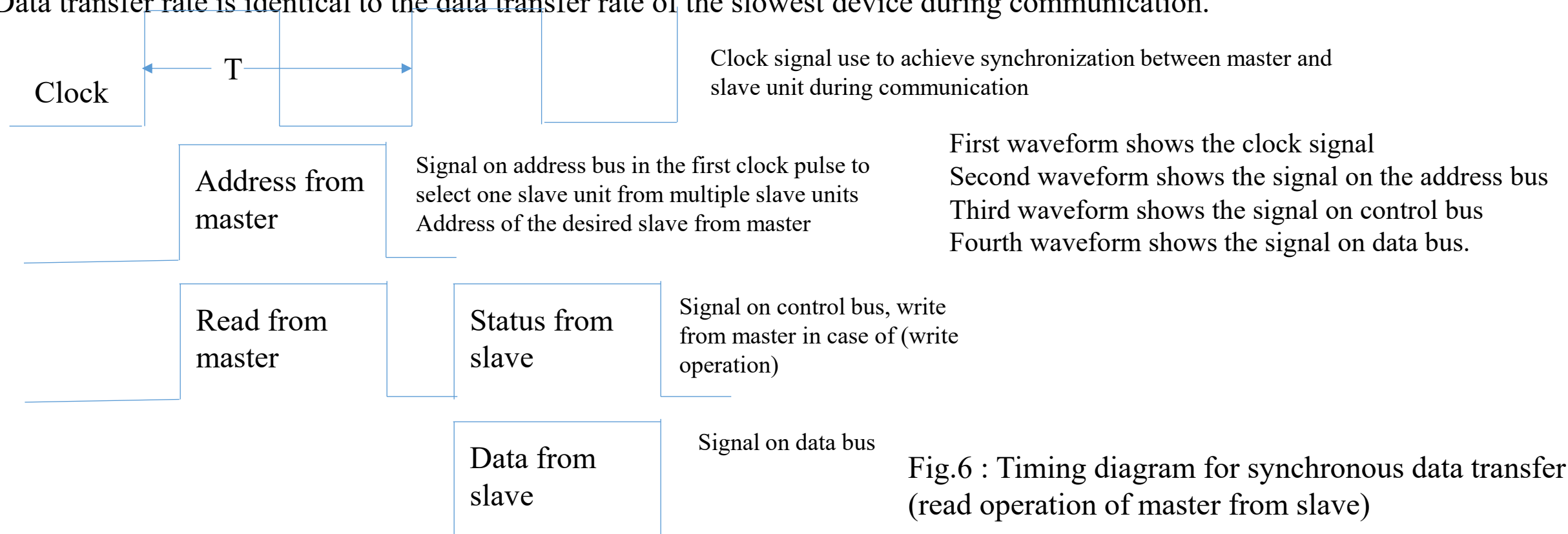
Fig.5 IO interface

Synchronous communication: Each item is transferred in a particular time slot which is known to both the source and destination.

Communication between the source and destination is synchronized. Such synchronization is achieved by using the same clock signal at the source and destination end. Separate clock signal having identical frequency may be used at source and destination end to achieve synchronization.

Useful for short distance communication as it is difficult to maintain identical clock signal (i.e. clock signals for source and destination are generated from the same clock generator) or different clock signal (i.e. clock signals for source and destination are generated from different clock generator) having identical frequency at the source and destination end for long distance communication.

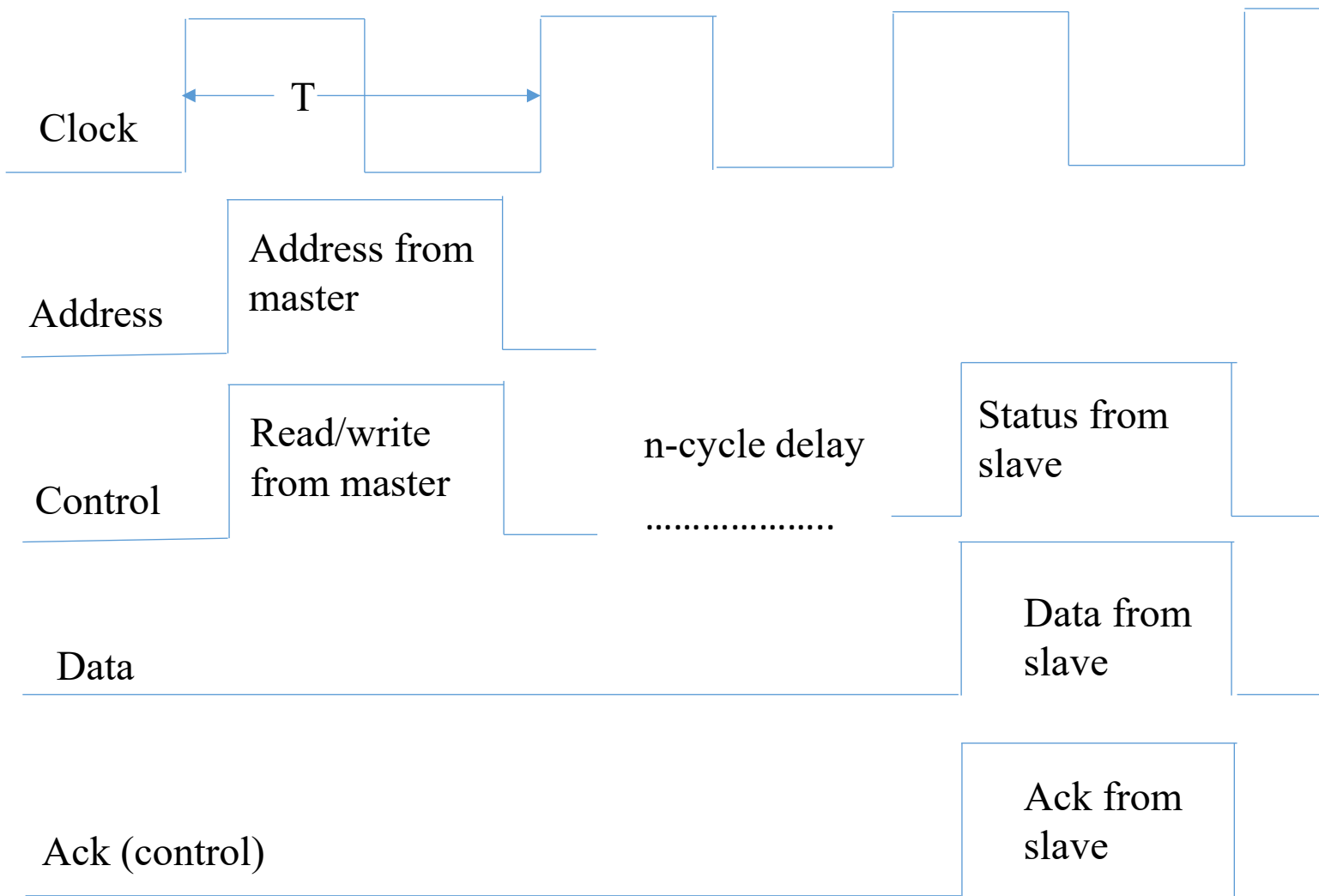
Data transfer rate is identical to the data transfer rate of the slowest device during communication.



In the first clock pulse all the slave units read address from the address bus and compare this address with its own address. Slave unit for which match occurs between the address on address bus and its own address reads the control signal from the control bus as placed by the master unit in the first clock pulse

In case of read operation from slave, this slave unit responds by sending status information on the control bus and data on the data bus in the second clock pulse

In case of write operation into slave, the master unit places data on the data bus in the second clock pulse. It can be shown in the fourth waveform by writing **data from master** instead of **data from slave**.



Slave unit responds by sending status information on the control bus, **status from slave** and data on the data bus, **data from slave** after a delay of amount **n clock cycles**. In such a case the master unit does not know exactly when the slave unit sends the requested data on the data bus during read operation of master unit from slave. So the slave unit also sends an acknowledgement signal, **Ack from slave** at the time of placing data on the data bus

Fig.7 : Timing diagram for read operation of the master unit from a slave unit when the slave unit is much slower than the master unit

Asynchronous communication: Such communication needs to exchange handshaking signal among source and destination. Devices having widely different data transfer rate can communicate asynchronously. Data transfer rate during communication depends upon the speed of communicating devices. Such communication needs more complex bus control circuitry. Useful for long distance communication.

One-way asynchronous data transfer: Timing signals are supplied by one of the two communicating devices to initiate data transfer operation.

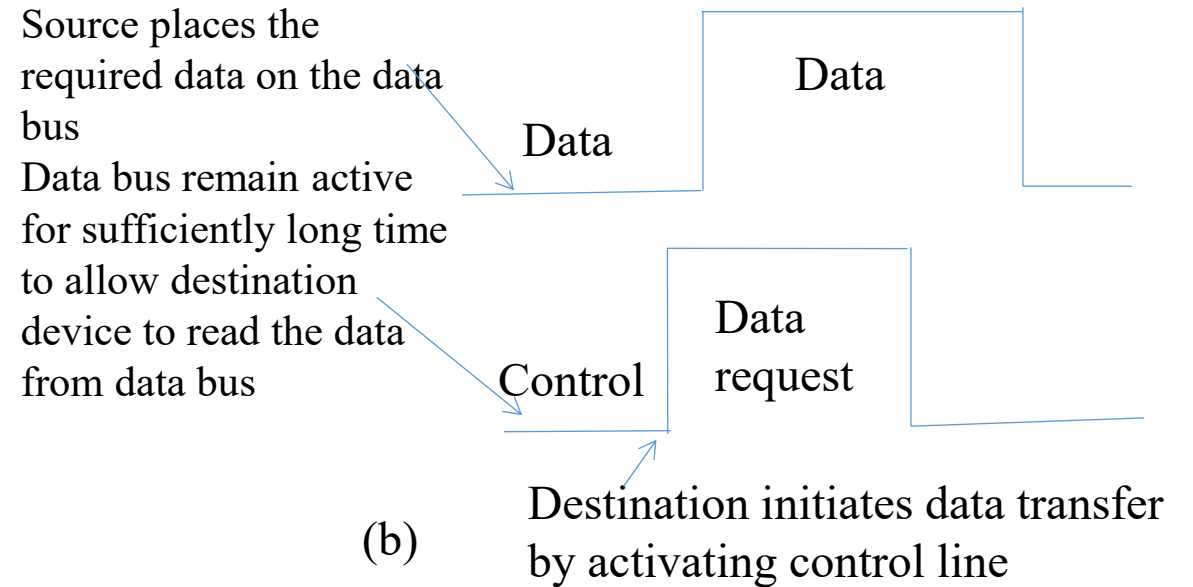
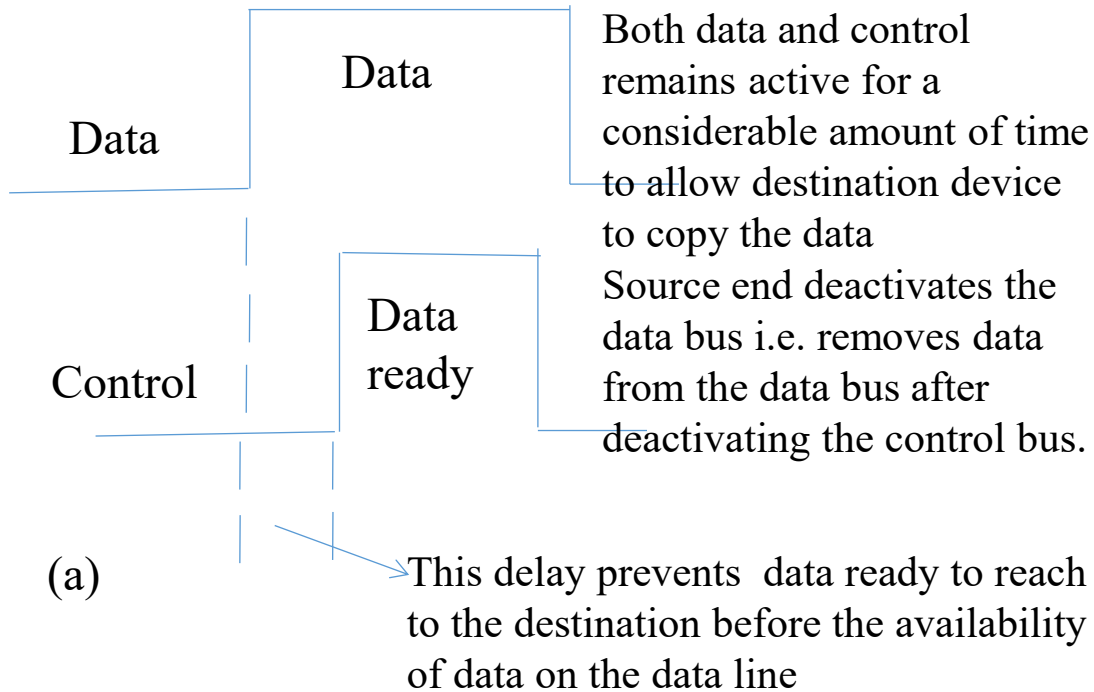


Fig. 8 Asynchronous 1-way data transfer (a) Source initiated data transfer (b) Destination initiated data transfer

The source end can't understand whether the data transfer operation is over successfully or not in case of one-way data transfer. Because the destination device may be unexpectedly slow in responding to the data ready signal of the source end, during that time the data may be lost. The two-way asynchronous data transfer is used to solve this problem.

Two-way asynchronous data transfer: Both the communicating devices (source and destination) generate timing signals during data transfer operation.

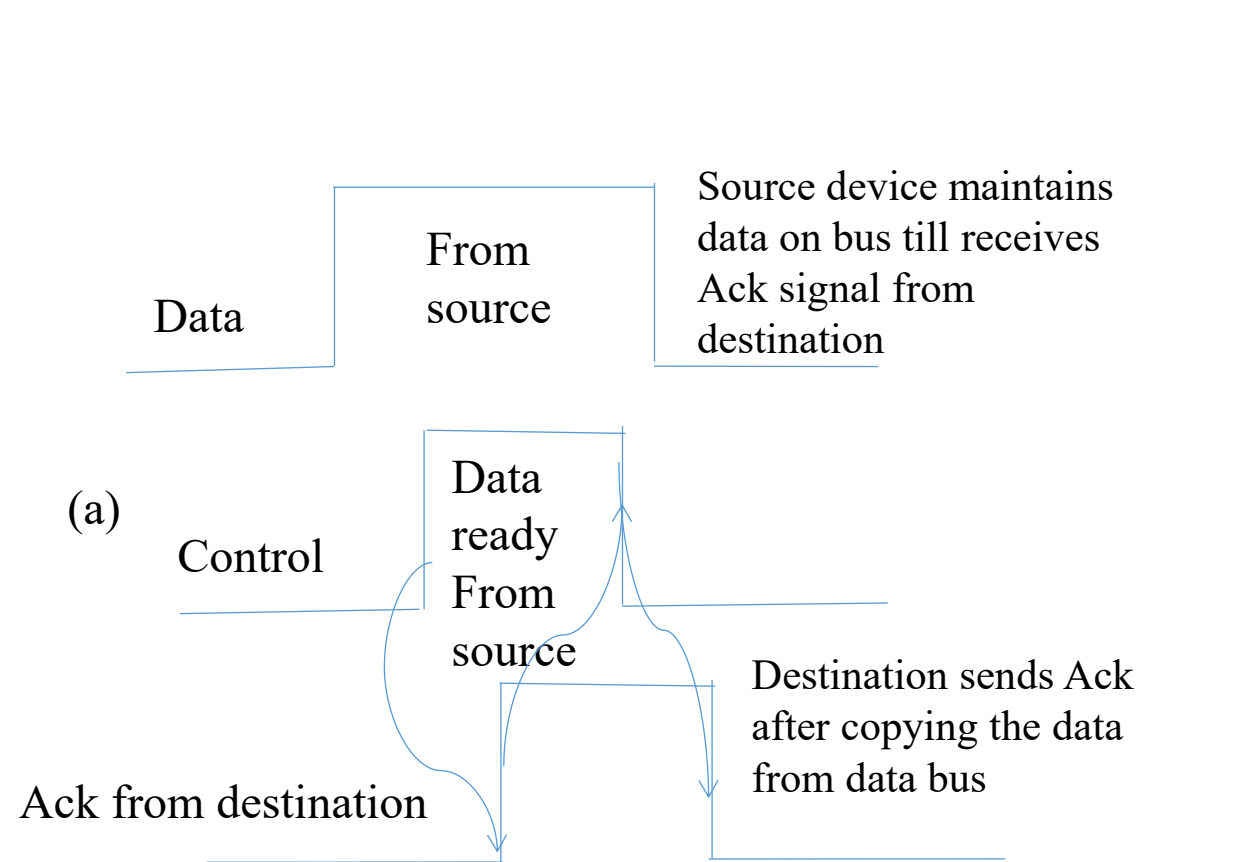
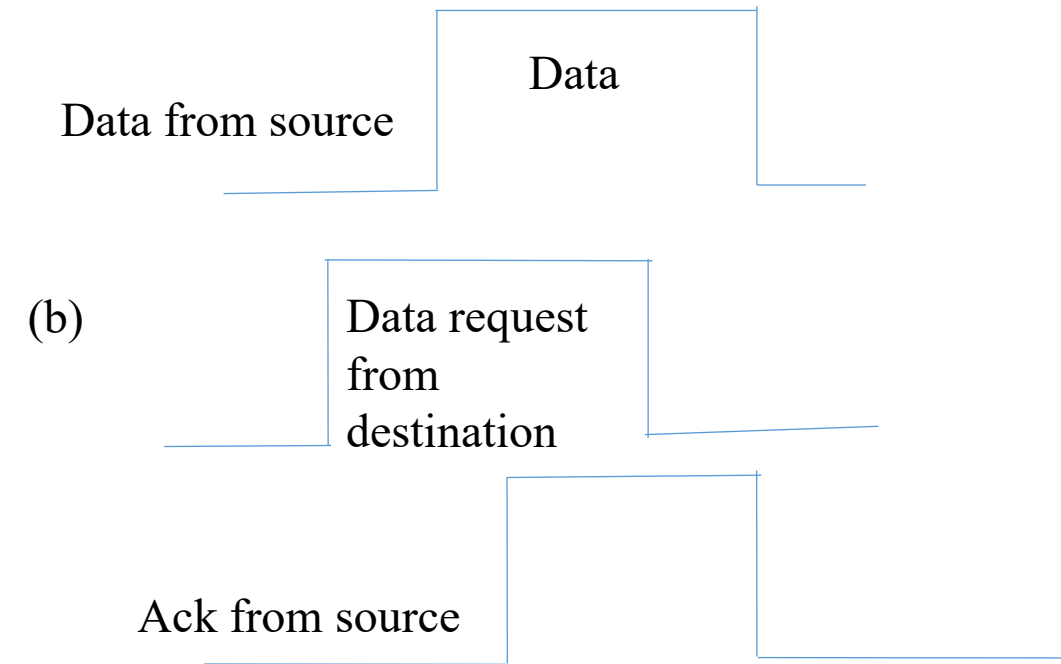


Fig. 9 Asynchronous 2-way data transfer (a) Source initiated data transfer (b) Destination initiated data transfer

Source maintains data on data bus till destination deactivates the data request line
Source unit deactivates data bus i.e. removes data from data bus after deactivating the control bus



Source unit activates Ack after placing requested data on data line

Source initiated data transfer:

Destination end reads the control signal from the control bus, copies the data from the data bus and sends acknowledgement signal to the source end

Source end deactivates the control bus i.e. removes the control signal (Data ready) from the control bus after receiving the acknowledgement signal from the destination end

Destination end deactivates the acknowledgement signal after the deactivation of the control signal by the source end

Source end deactivates the data bus i.e. removes data from the data bus after deactivating the control bus as in one-way data transfer.

Destination initiated data transfer:

Destination end initiates data transfer by placing a control signal on the control bus, **Data request from destination**

Source end places the data on the data bus, **Data** and sends acknowledgement signal, **Ack from source** after placing the data on the data bus to the destination end to inform the destination end that the data is ready on the data bus

Destination end copies the data from the data bus after receiving the acknowledgement from the source end and deactivates the control bus i.e. removes data request from the control bus.

Source end maintains the data on the data bus till the destination end deactivates the data request signal on the control bus.

The use of acknowledgement signal both in source initiated data transfer and destination initiated data transfer helps the source end to verify the successful data reception at the destination end and hence solves the problem of one-way data transfer.

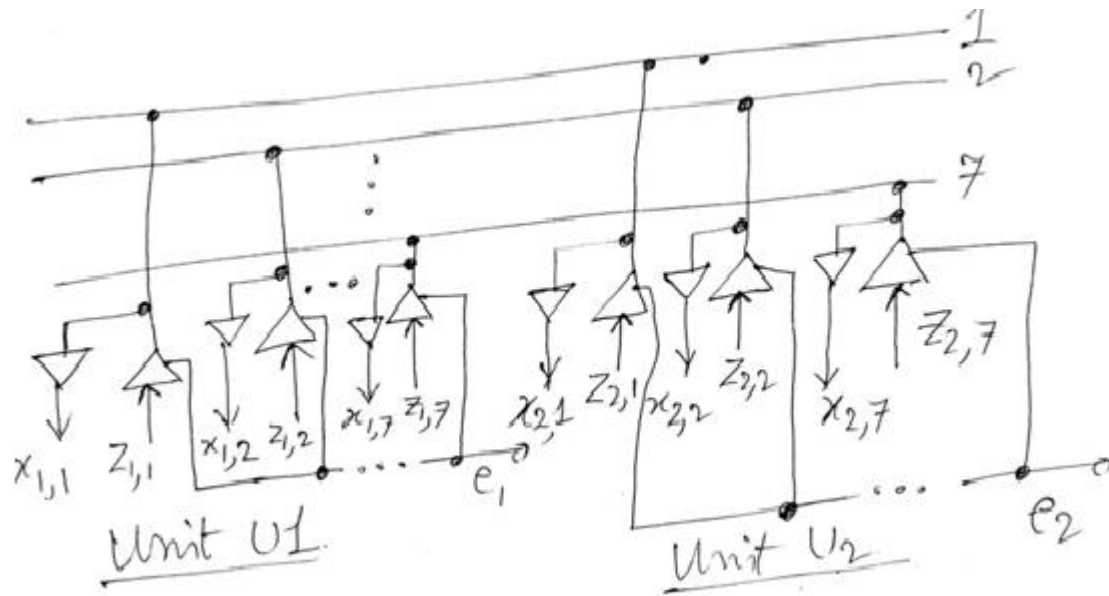


Fig 3 Shared bus

Seven bidirectional bus lines (1234567) are used for data input as well as data output.

$Z_{1,1}, Z_{1,2}, \dots, Z_{1,7}$ are the 7 data inputs lines and $x_{1,1}, x_{1,2}, \dots, x_{1,7}$ are the 7 data output lines of unit U1.

$Z_{2,1}, Z_{2,2}, \dots, Z_{2,7}$ are the 7 data inputs lines and $x_{2,1}, x_{2,2}, \dots, x_{2,7}$ are the 7 data output lines of unit U2.

Seven tristate gates (drivers) are used in the seven input lines of unit U1 and U2.

Control signal for the tristate gates of unit U1 is e_1 and U2 is e_2 . So if $e_1=1$ all the 7 tristate gates in unit U1 are enabled and if $e_2=1$ all the 7 tristate gates in unit U2 are enabled.

Seven receivers are used in the seven output lines of unit U1 and U2 for receiving data from bus during read operation.

Use of tristate logic for interfacing two units greatly reduce fan-in and fan-out constraints on bus lines, facilitate bidirectional signal transmission over a bus line by allowing the same bus line to serve both as input and output.

Tristate logic helps use the same set of bus for input as well as output which in turn helps to minimize the cost of system bus.

Operation of the circuit in Fig 3 for communication between U1 and U2

(i) If $e1=1$ and $e2=0$

All the 7 tristate gates in unit U1 are enabled. These 7 tristate gates transfer the data from the 7 input data lines ($z_{1,1}, z_{1,2}, \dots, z_{1,7}$) of unit U1 to the 7 bidirectional bus lines (1234567). That is the unit U1 performs write operation in the bus (1234567), in effect making $x_{2,i}=z_{1,i}$ for all i ($i=1$ to 7).

All the 7 tristate gates in unit U2 are disabled as $e2=0$. So the unit U2 can't perform write operation in the bus (1234567). But the unit U2 can perform read operation from the bus (1234567) through receivers.

(ii) If $e1=0$ and $e2=1$

U2 drives the bus, and information is transferred over the bus from U2 to U1, in effect making $x_{1,i}=z_{2,i}$ for all i ($i=1$ to 7).

U2 performs write operation in 1, 2, 3, 4, 5, 6, 7 from $z_{2,1}, z_{2,2}, z_{2,3}, z_{2,4}, z_{2,5}, z_{2,6}, z_{2,7}$ and the unit U1 performs read operation from 1, 2, 3, 4, 5, 6, 7 to $x_{1,1}, x_{1,2}, x_{1,3}, x_{1,4}, x_{1,5}, x_{1,6}, x_{1,7}$.

(iii) If $e1=0$ and $e2=0$

The outputs of both U1 and U2 are logically disconnected from the bus.

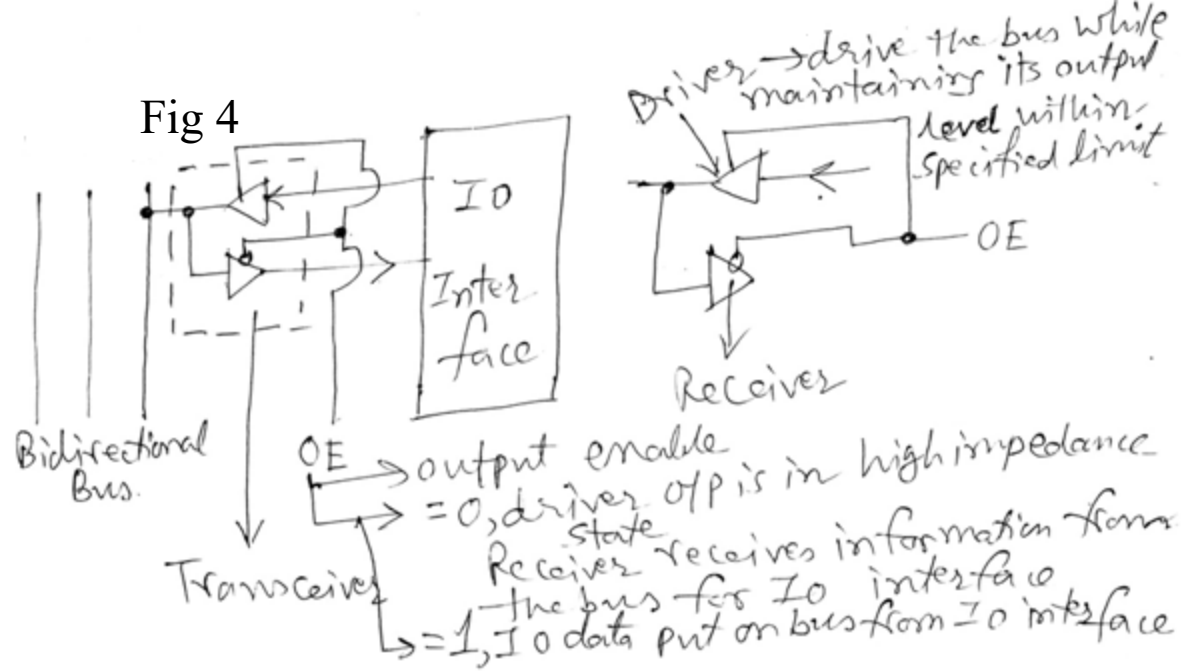
Neither the unit U1 nor the unit U2 performs write operation in the bus (1234567) as all the 7 tristate gates both in unit U1 and unit U2 are disabled.

(iv) If $e1=1$ and $e2=1$

It is invalid, since it applies two different signals to each bus line making the resultant bus state indeterminate.

It allows both the units to perform write operation into the bus simultaneously. So it is invalid as the bus state becomes indeterminate.

Fig 4



Advantage of using transceiver:

Reduces fan in and fan out constraints on the bus lines.

Facilitates bidirectional signal transmission over a bus line by allowing same line to be used as input and output.

At a time one device can perform write operation in the bus but all the devices can perform read operation from the bus.

Transceiver: Fig 4 shows transceiver between the bidirectional bus and IO interface

Transceiver operates both in transmitting mode and receiving mode.

Driver part of the transceiver is enabled at the time of write operation by the IO device to the bidirectional bus

Receiver part of the transceiver is enabled at the time of read operation by the IO device from the bus through the IO interface.

Driver is positive controlled but the receiver is negative controlled.

Control input is output enable (OE).

If $OE=0$ the driver output is in high impedance state but the receiver is enabled, receiver receives the data from bus and sends the data to the IO device through the IO interface circuit.

If $OE=1$ the receiver output is in the high impedance state but the driver is enabled, driver puts data on the bus from the IO device through the IO interface circuit.

Arbitration:

Several master and/or slave devices are connected to a shared bus.

More than one such devices may generate request for accessing the bus at the same time.

Bus access can be given to a single device at a time.

So a selection mechanism called bus arbitration is required to select one device from several requesting devices.

Daisy chaining, polling, independent requesting are the three bus arbitration schemes.

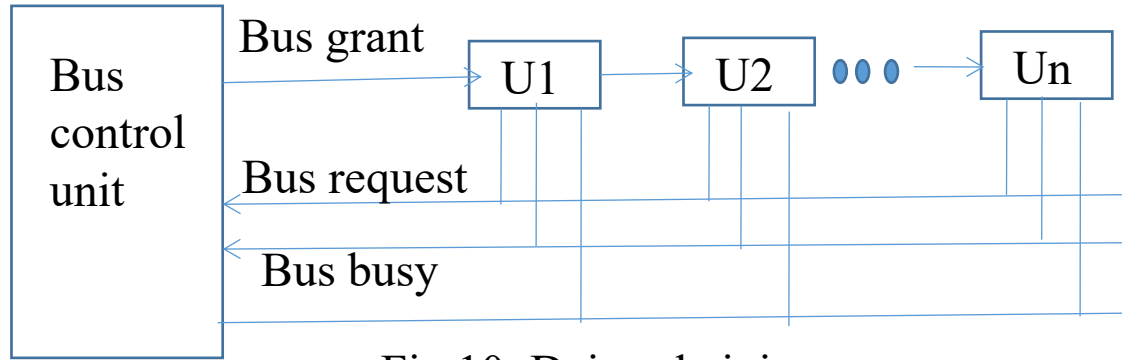


Fig 10: Daisy chaining

Daisy chaining:

Bus request, Bus busy, Bus grant are the three control signals.

All the bus devices (U1, U2...Un) are connected to a common bus request line.

Bus request line becomes active when one or more devices are requesting use of the bus.

Bus control unit places a signal on the bus grant line in response to bus request if bus busy is inactive.

Bus grant line is connected serially from device to device.

First requesting device receives the bus grant signal, blocks further propagation of bus grant signal, enables its physical bus connection, activates bus busy and begins to use the bus, deactivates bus busy when its bus activity is over.

A non-requesting device forwards the bus grant signal to the next device. Thus if two devices are simultaneously requesting bus access, the device closest to the bus control unit receives the bus grant signal first and gains access to the bus.

Selection priority is therefore completely determined by the order in which the devices are linked together (chained) by the bus grant line.

Requires very few control lines and employs a very simple arbitration algorithm.

Can be used with an essentially unlimited number of devices.

Priority of each device cannot be changed.

If device U_i (i^{th} device, $1 \leq i \leq n$) is unable to propagate the bus grant signal due to failure, then all $\{U_j\}$ where $j > i$ cannot gain access to the bus.

Now let us consider U_n is the only requesting device and the other $(n-1)$ number of devices are non-requesting. In such a case U_n has to wait till it receives bus grant from $(n-1)^{\text{th}}$ device. So the selection priority i.e. the priority of selecting a device out of multiple requesting devices depends upon the physical position of the device.

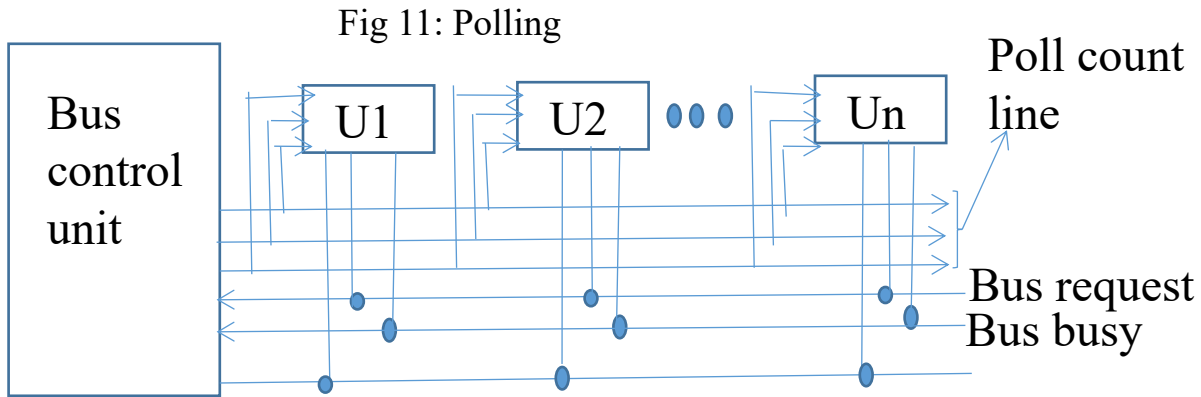
Polling:

Bus grant line is replaced by a set of lines called poll count lines which are connected directly to all devices ($U_1, U_2, \dots, U_n$) on the bus.

All the bus devices are connected to a common bus request line.

Bus control unit proceeds to generate a sequence of numbers on the poll count lines in response to a signal on bus request. Each requesting device compares its unique address with the number on the poll count lines. The requesting device for which a match occurs between the unique address and the number on the poll count lines activates bus busy and begins to use the bus.

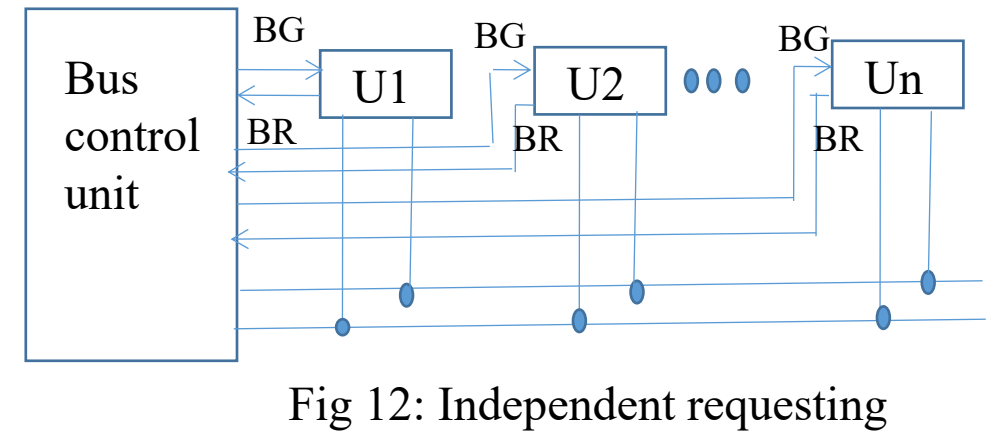
Bus control unit responds by terminating the polling process.



Priority of a bus device is determined by the position of its address in the polling sequence. This sequence is normally programmable and hence the selection priority can be altered under program control.

A failure in one device does not affect any of the other devices. This flexibility is achieved at the cost of more control lines. If the number of device is n , then $\log_2 n$ number of poll count lines are required instead of one bus grant line.

Number of devices that can share the bus is limited by the addressing capability of the poll count lines.



Independent requesting:

Each device has its own bus request (BR) and bus grant (BG) line.

Bus control unit identifies the requesting device immediately after receiving the bus request from it. Priority is determined by the bus control unit and may be programmable.

A failure in one device does not affect any of the other devices.

$2n$ bus request and bus grant lines must be connected to the bus control unit in order to control n devices. In contrast, daisy chaining requires 1 bus request and 1 bus grant line, while polling requires approximately $\log_2 n$ lines.

UNIBUS system:

Combination of daisy chaining and independent requesting.

m bus request lines having different priority are connected to the bus control unit.

n number of devices are distributed among **m** bus request lines depending upon their priority.

One bus grant line is used corresponding to each bus request line to select one device from multiple requesting devices like daisy chaining.

Number of control line is **2m (m bus request +m bus grant)**.

Failure of one device affects other devices connected to the same bus request line.

Input-Output system:

Such system transfers information between MM or CPU and IO devices.

Includes IO devices, control units for these devices, software designed to carry out IO operation.

Input/output controller in the control unit of IO device connects the device to the bus system of CPU.

Typically communicates with CPU and MM over the system bus.

CPU initiates the control by sending IO requests for read/write operation.

Input/output controller receives such requests from CPU and sends device-specific control signals to the device for performing the requested read/write operation.

Such controller also manages the data flow to or from the device.

Input/output controller is sometimes called device controller.

Software that directly interacts with a device controller to carry out IO operation is called a device driver.

IO operation: Includes data transfer between IO device and MM, data transfer between IO device and CPU.

Objective is communication with the external world.

Programmed IO:

IO device does not have direct access to MM.

Programmed IO is a method of controlling IO operation.

In programmed IO all IO operations are executed under the direct control of CPU.

Function of CPU

Executes program to transfer data between the buffer register connected to the IO device and MM.

This program includes

Input instruction to transfer data from the input device to CPU register (i.e. accumulator) and a store instruction to transfer data from CPU register to MM.

Load instruction to load data in CPU register from MM and an output instruction to transfer data from CPU register to output device.

Includes instruction to compute MM address for performing read/write operation and data word counting (i.e. to count the number of bytes to be transferred between IO device and MM).

Determines IO status i.e. identifies the IO device that is ready for data transfer operation.

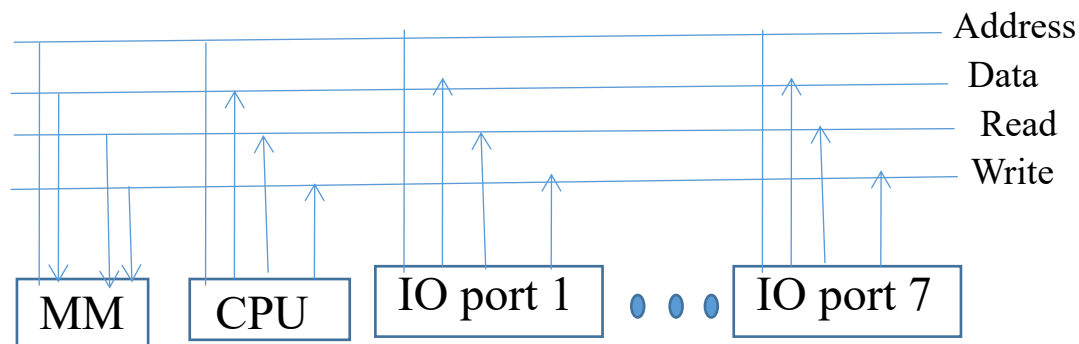


Fig 13: Memory mapped IO

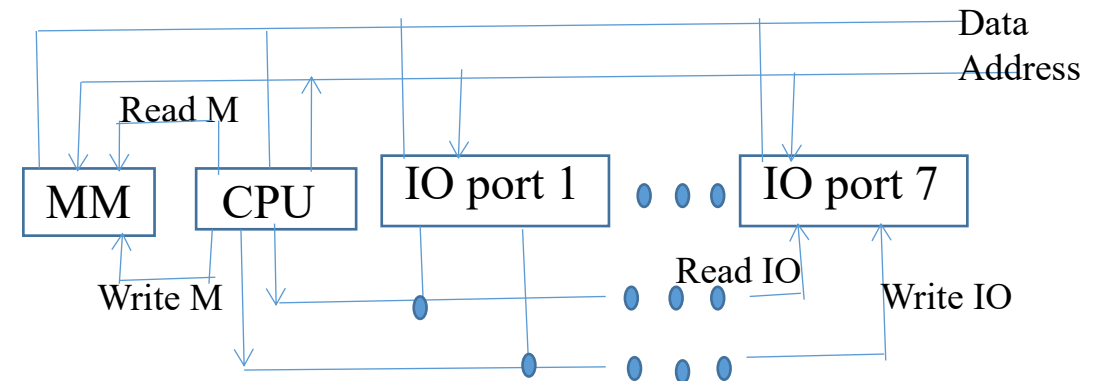


Fig 14: IO mapped IO

Memory-mapped IO: Fig 13 shows memory-mapped IO.

Programmed IO with shared MM and IO address space. [**Note**: For example, the address space for 16-bit address is 2^{16} . A few locations out of these 2^{16} locations are used for MM and the other locations are reserved for IO devices. So the address space of amount 2^{16} is shared among MM and IO.]

System bus includes

Address bus, data bus, control bus.

MM, CPU, IO devices share the same system bus.

Each junction between the system bus and the IO device is called an IO port. Seven such IO ports are shown in Fig 13. A unique address is assigned to each IO port. The IO port includes a data buffer register for storing data temporarily during read/write operation.

Suppose CPU generates an address X in the address bus.

Address on the address bus, control signal (Read/Write) on the control bus, data on the data bus activate MM if the address X is an address of the address space which is used by MM. That is read and write control lines are activated by CPU to initiate a memory access cycle (MM read/write).

Address on the address bus, control signal (Read/Write) on the control bus, data on the data bus activate IO device if the address X is an address of the address space which is reserved for IO device. That is read and write control lines are activated by CPU to initiate IO access cycle (IO read/write).

Memory load/store instructions are used to transfer a word to or from an IO port.

No special IO instructions are required.

[**Note**: Move A, X moves data from location X in the address space to accumulator. This data transfer operation is in between MM and CPU if the address X is used for MM. This data transfer operation is in between an input device and CPU if the address X is reserved for IO.]

IO-mapped IO: Fig 14 shows IO-mapped IO.

Programmed IO with separate MM and IO address space.

Seven IO ports are shown in Fig 14.

IO-mapped IO has separate control signal for MM (written as **Read M** for read operation from MM and **Write M** for write operation into MM) and IO (written as **Read IO** for read operation from input device and **Write IO** for write operation into output device).

Suppose CPU generates an address X in the address bus and activates Read M then X is the address of a MM location and CPU wants to perform read operation from address X of MM.

But if CPU generates an address X in the address bus and activates Write IO then X is the port address of an output device and CPU wants to perform write operation in an output device whose port address is X.

Separate IO reference instructions activate Read IO/Write IO. [**Note**: For example, IN <port address> activates Read IO to transfer data from an input device whose address is equal to the port address as mentioned in the instruction to accumulator and OUT <port address> activates Write IO to transfer data from accumulator to an output device whose address is equal to the port address as mentioned in the instruction.] whereas the usual memory load/store instructions activate Read M/Write M. [**Note**: Move A, X activates Read M as it is a MM read operation from location X to A and Move X, A activates Write M as it is a MM write operation from A into location X.]

Disadvantage of programmed IO:

Data transfer between MM and IO devices is under the control of CPU as CPU has to execute data transfer instructions like Move, IN, OUT.

Let the size of accumulator (CPU register A) is 8-bit. So only 8-bit data can be transferred at a time. For example, the execution of the instruction IN <port address> transfers 8-bit data from an input device to accumulator as the size of accumulator is 8-bit. So to transfer N-byte of data from input device to MM through accumulator, CPU has to execute two instructions (one instruction to transfer data from input device to accumulator and another instruction to transfer the same data from accumulator to MM) N times.

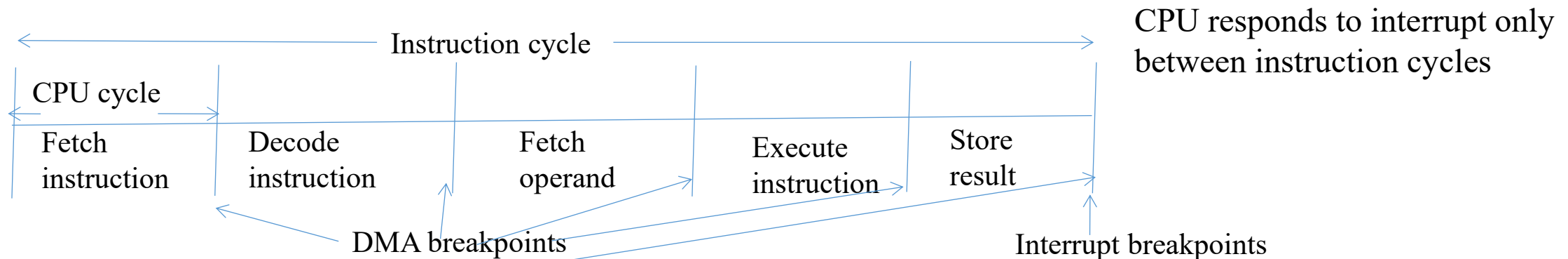
CPU remains busy for data transfer operation instead of executing the normal program.

Moreover, CPU has to determine the IO status i.e. to identify the IO device that is ready for data transfer operation.

Solutions of the difficulties of programmed IO:

Interrupt driven data transfer and direct memory access (DMA) are the two other methods of data transfer between MM and IO device to solve the difficulties of programmed IO.

Fig 15 shows one instruction cycle which is divided into 5 CPU cycles. Several CPU cycles use system bus during the execution of an instruction. One interrupt break point is shown at the end of the instruction cycle whereas one DMA breakpoint is shown at the end of each CPU cycle.



Interrupt driven data transfer:

IO device sends a signal called interrupt request to the CPU after becoming ready for data transfer operation.

Interrupt request line of CPU becomes active.

Suppose CPU receives this interrupt request during the execution of i^{th} instruction in the main program.

Program counter (PC) contains the address of $(i+1)^{\text{th}}$ instruction.

CPU completes the execution of i^{th} instruction as interrupt breakpoint is at the end of an instruction cycle as shown in Fig 15 , stores the content of PC in **temporary storage** to resume the execution of main program from $(i+1)^{\text{th}}$ instruction, saves other CPU status [**Note:** the condition of various flags like carry flag, sign flag, zero flag etc. is an example of such CPU status, whether set or reset after the execution of i^{th} instruction], sends interrupt acknowledgement to the interrupted device to deactivate the interrupt request line.

Deactivation of interrupt request line helps CPU to become ready for receiving the next interrupt request from the other IO devices.

CPU loads PC by the starting address of the interrupt service routine (ISR) and starts to execute ISR for data transfer operation between MM and IO device

CPU resumes the execution of main program from instruction $(i+1)$ when return instruction is encountered in ISR.

Return instruction loads PC by the address of the $(i+1)^{\text{th}}$ instruction from the **temporary storage**.

Difference between subroutine call and ISR:

In case of subroutine		In case of ISR
Main program	FUNC subroutine	Main program
I1	I6	I1
I2	I7	I2
I3 Call FUNC	I8	I3
I4		I4
I5		

Subroutine call: I1, I2, I3, I4, I5 are the instructions in the main program.

I3 is calling a subroutine called FUNC.

I6, I7, I8 are the instructions in the subroutine FUNC.

Suppose FUNC is written by the programmer to find the square root of X. Call FUNC instruction in the main program sends the value of X to FUNC, FUNC computes square root of X and returns the result of computation to the main program.

After this the instructions I4 and I5 in the main program are executed.

So the subroutine FUNC is used by the main program to calculate square root of X, main program sends X to the FUNC and FUNC is written by the programmer.

ISR: I1, I2, I3, I4 are the instructions in the main program.

Suppose CPU receives an interrupt from the external IO device during the execution of instruction I2 in main program.

CPU completes the execution of I2 and starts to execute ISR till return instruction is encountered in ISR. CPU executes the instructions I3 and I4 in the main program after returning to the main program from ISR.

Main program is written by the programmer but ISR is not written by the programmer.

ISR is not used by the main program.

CPU executes ISR in response to the received interrupt of the IO device to initiate data transfer operation.

So main program and ISR are the two independent program.

Main program does not send any data like to ISR.

Enabling and disabling interrupt:

Interrupt request comes from an external device and may come at any time from different devices to CPU.

Interrupt request from any device stops the execution of one program and starts the execution of another program.

So it is required to control the interruption of program execution carefully.

Disable interrupt (DI) instruction is used as the first instruction of ISR to ensure no further interruption until the enable interrupt (EI) instruction is executed.

EI is the last instruction of ISR before the return instruction to guarantee that the execution of ISR is completed before further interruption.

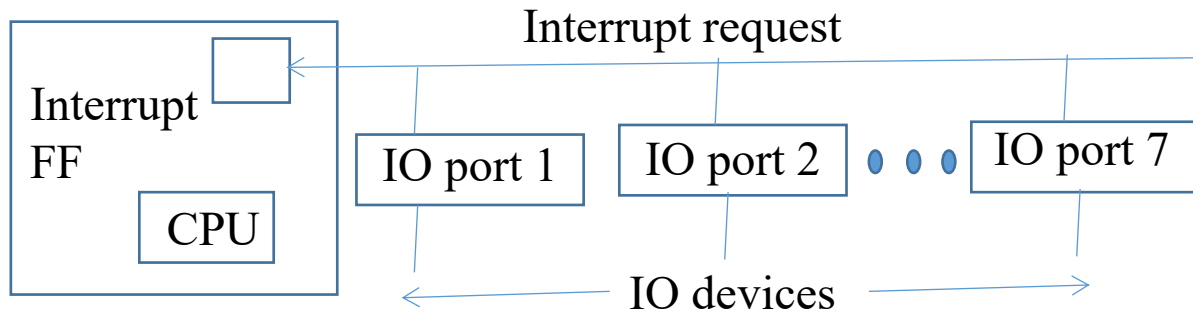


Fig 16: Single line interrupt system

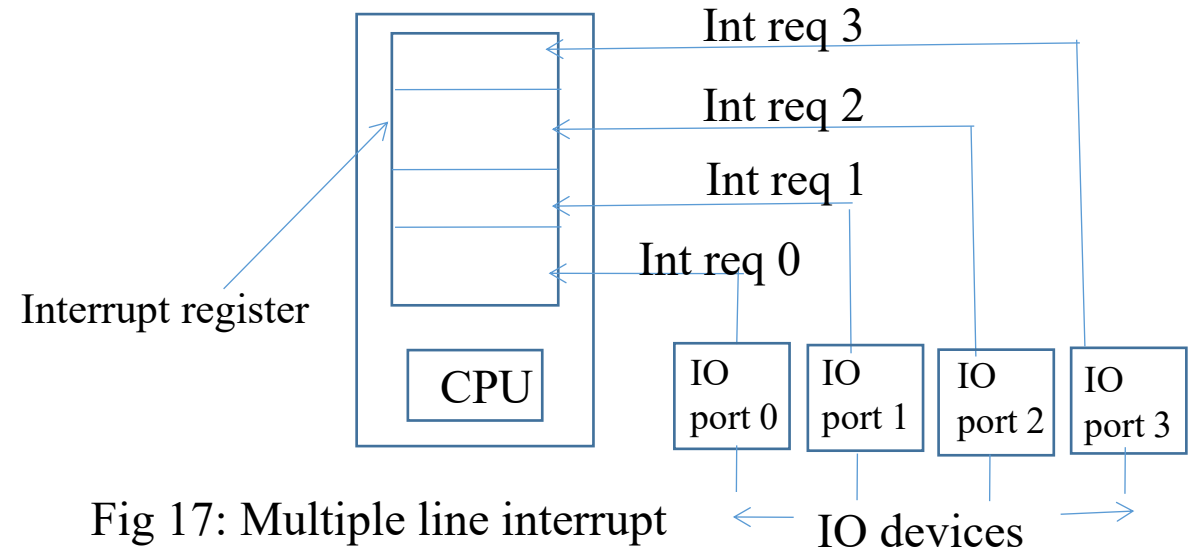


Fig 17: Multiple line interrupt

Interrupt selection:

In Fig. 16, 7 IO ports are connected to the interrupt request line.

A single interrupt request line is shared among 7 number of IO devices for sending interrupt request.

CPU sends interrupt acknowledgement (INTA) in response to the interrupt request.

INTA signal is propagated serially through all the devices starting from nearest one as in daisy chaining.

Fig 17 shows multiple line interrupt system having 4 IO devices.

Each device has its own interrupt request line for sending interrupt request to CPU as in independent requesting.

CPU stores the received interrupts in an interrupt register.

Source of interrupt is immediately known to CPU.

Each interrupt request line has a unique priority.

CPU sends INTA to the interrupted devices depending upon the priority of the interrupt request in the interrupt register.

Disadvantage of interrupt driven data transfer:

CPU may have to execute a program to fetch the starting address of ISR before starting its execution.

This problem is solved in vectored interrupt in which an IO device sends a special code to CPU along with interrupt request to compute the starting address of ISR.

In programmed IO CPU checks the ready pin of IO devices to identify the IO device that is ready for data transfer operation.

It consumes a considerable amount of CPU time. It is eliminated in interrupt driven data transfer as a device sends an interrupt to CPU as soon as it becomes ready for data transfer operation.

But in interrupt driven data transfer CPU remains busy during data transfer operation between IO device and MM due to the execution of ISR like programmed IO.

This problem is solved in DMA mode of data transfer.

DMA mode of data transfer:

IO device sends data transfer request to the DMA controller.

Fig 18 shows the communication between CPU and DMA controller after receiving data transfer request from IO device.

System bus includes address, Dout (data out), Din (data in).

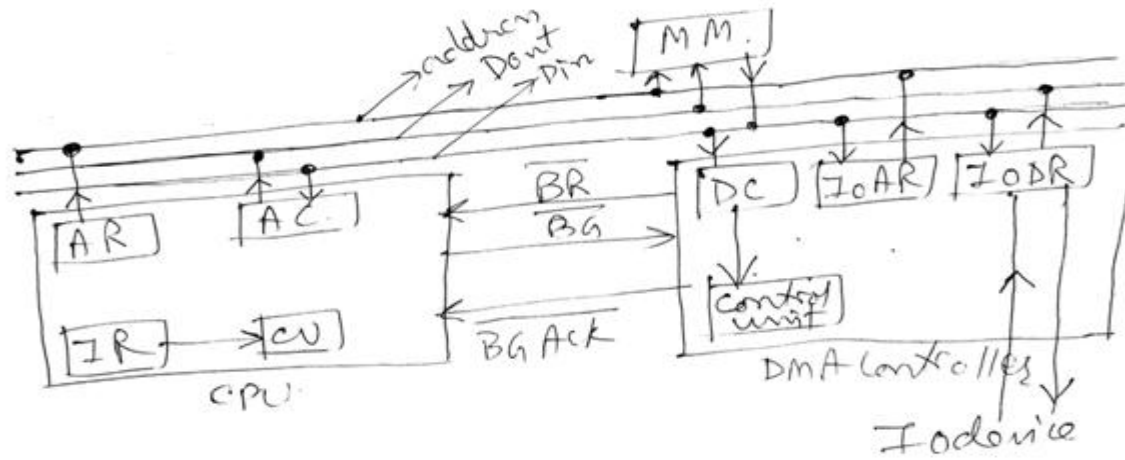
Inside the block of CPU AR is address register, AC is accumulator, IR is instruction register and CU is control unit.

Inside the block of DMA controller DC is data count, IOAR is the address register and IODR is the data buffer register.

IODR is connected with IO device and data bus (Dout and Din as shown in Fig 18).

IODR stores data during data transfer from IO device to MM through Dout and MM to IO device through Din.

DMA controller receives data transfer request from IO device and activates bus request line (BR) to release buses for data transfer between MM and IO device.



CPU releases system bus at the end of current CPU cycle (i.e. at the DMA breakpoint).

CPU sends DMA parameters like **starting address** of MM from where data transfer operation takes place, **number of words/bytes** to be transferred between MM and IO device, direction of data transfer (IO device to MM or MM to IO device) to the DMA controller through data bus.

CPU activates bus grant line (BG).

DMA controller stores the **starting address** in IOAR, word or byte count i.e. the **number of words or bytes** to be transferred in DC, activates bus grant acknowledgement () and deactivates .

Data transfer takes place between MM and IO device under the supervision of DMA controller.

DMA controller provides memory address and all the bus signals, that control data transfer operation.

DMA controller updates IOAR and DC after transferring each word/byte.

DMA controller raises an interrupt signal to inform CPU about the end of data transfer operation and returns the system bus to CPU by deactivating

Modes of data transfer using DMA: Data can be transferred in several different ways under DMA control.

DMA block transfer

Transfers a data word sequence of arbitrary length in a single continuous burst while the DMA controller is the master of the system bus.

This mode supports maximum IO data transmission rate while CPU may remain inactive for relatively long period.

Cycle stealing

Allows DMA controller to use the system bus to transfer one or several data words after which it must return control of the system bus to CPU.

That is a few CPU cycles are stolen by the DMA controller as CPU remains inactive during the data transfer operation by DMA controller

It needs a sequence of DMA bus transaction to transfer long blocks of IO data.

It reduces IO data transfer rate and interference of CPU activity than block transfer.

It is possible to eliminate the CPU interference completely by designing the DMA interface so that bus cycles are stolen only when the CPU is not actually using the system bus; this is transparent DMA.

Access time of disk:

Total time required for a storage device (such as a Hard Disk Drive) to process a read/write request and transfer the requested data, spanning from the initial request to the moment the data is available

Access time = Seek time + Rotational latency + Transfer time

RAID (Redundant Array of Independent Disks)

Provides significant advantages over a single hard disk drive

Key advantages of RAID over a single disk include:

Improved Performance (Speed): RAID uses "striping" to distribute data across multiple disks, allowing read and write operations to happen in parallel, significantly increasing throughput compared to a single drive.

Data Redundancy and Fault Tolerance: RAID mirrors or stripes data with parity, allowing the system to continue operating without data loss if one or more drives fail.

Increased Storage Capacity: By pooling several smaller, less expensive disks, RAID allows for the creation of a large, high-capacity logical drive.

Improved System Uptime: Because RAID provides high availability, servers can continue to run and access data even during a disk failure, preventing downtime.

Higher Data Reliability: RAID configurations like RAID 5 and 6 enhance data integrity, reducing the probability of total data loss compared to relying on a single disk

RAID 0 (Striping)

Array has multiple disks, let the number of disks in the array is N

Each disk operates independently

Write operation:

Breaks a file into N number of small pieces of data

Stores the N small pieces of data in N different disks

Read operation:

All the N number of disks deliver data in parallel during read operation. All these N pieces of data are stored in a buffer during read operation to reassemble the file from N small pieces of data

Access time: Seek time + Rotational latency time + Data transfer time

The seek time and rotational latency time to locate the beginning of the piece of data on each disk are not reduced i.e. identical to the use of single disk for storing the same file.

The data transfer time reduces as it is equal to the time which is required to read the entire file from a single disk divided by the number of disks used in the array i.e. N

This configuration of RAID is the simplest possible disk array operation in which the data transfer time component of access time reduces and hence the data access time during read operation reduces

RAID 1 (Mirroring)

Array has multiple disks, let the number of data disks in the array is N

Each disk operates independently

Write operation:

Breaks a file into N number of small pieces of data

Stores the N small pieces of data in N different data disks

Stores identical copies of data on two disks rather than on a single disk that is each data disk has a redundant disk. If one disk drive fails all read and write operation from this faulty disk are directed to its redundant disk to avoid any data loss and hence improves reliability

Read operation:

All the N number of disks deliver data in parallel during read operation. All these N pieces of data are stored in a buffer during read operation to reassemble the file from N small pieces of data
Costly way to improve reliability as it needs $2N$ number of disks (N data disks + N redundant disks)

Four disks of RAID 2 configuration is shown in Fig 1. The left two disks are the data disks (DD 1 and DD 2 as written at the top of these disks) whereas the right two disks are the ECC (error correcting code) disks (ECC 1 and ECC 2) in Fig 1

Write operation

Strips the data bits (b1, b2, b3, b4, b5, b6) in the two data disks
[File is divided into bits while storing the entire file in the data disks]
Calculates ECC (using hamming code) during each write operation and stores the bits (E1, E2, E3, E4, E5, E6) of ECC in the ECC disks

Read operation

Reads the data bits from the data disks and computes **new ECC** from these data bits
Reads the bits of ECC from the ECC disks and compares these bits with the bits of **new ECC** to verify the data bits for reliability

Uses lot of disks (Data disks + ECC disks) and hence difficult to maintain synchronization among the disks during read write operation
Good for sequential reading and fair for sequential writing as ECC calculation is required even if the write operation is performed in a single bit
Random read is fair but random write is slow due to the overhead of ECC calculation and bit stripping
Single bit error can be detected and corrected easily, multiple bit error can be detected but not corrected

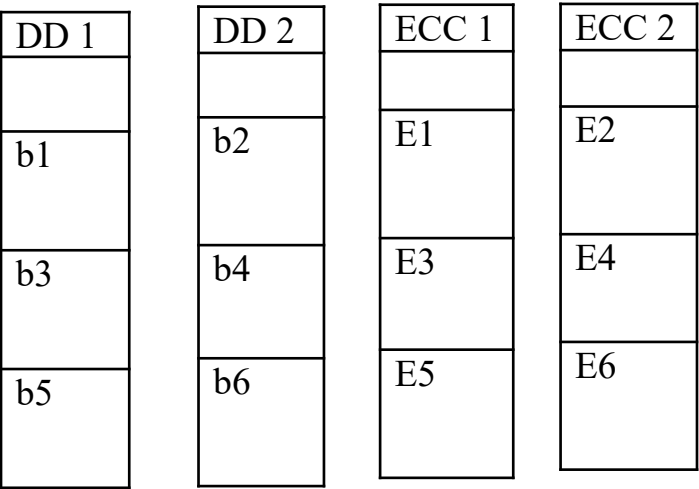


Fig 1: RAID 2

Fig 2: RAID 3

DD 1	DD 2	DD 3	PD
B1	B2	B3	P1
B4	B5	B6	P2
B7	B8	B9	P3

Four disks of RAID 3 configuration are shown in Fig 2. The left three disks are the data disks (DD 1, DD 2, DD 3) whereas the right most disk is the parity disk (PD) in Fig 2

Write operation

Strips the data bytes (B1, B2, B3, B4, B5, B6, B7, B8, B9) in the three data disks

[File is divided into bytes instead of bits (done in RAID 2) which reduces the required number of data disks in RAID 3 than RAID 2 to store the entire file]

Calculates parity byte corresponding to each row of data byte during each write operation and stores the parity byte in the parity disk

[For example, the parity byte P1 is calculated from the data bytes B1, B2, B3; P2 is calculated from the data bytes B4, B5, B6; P3 is calculated from the data bytes B7, B8, B9]

Read operation

Reads the data bytes from the data disks and computes **new parity byte** from these data bytes

Reads the parity byte from the parity disk and compares this parity byte with the **new parity byte** to verify the data bytes for reliability

$N+1$ (N data disks + 1 parity disk, $N=3$ in Fig 2) is the number of disks

If a data disk fails, the data bytes of this faulty disk can be recovered by an appropriate calculation between the parity bytes and the data bytes of the remaining data disks

High read speed but low write speed due to parity byte calculation for each write operation

Single disk error can be detected and corrected easily

Fig 3: RAID 4

DD 1	DD 2	DD 3	PD
A	B	C	P1
D	E	F	P2
G	H	I	P3

Four disks of RAID 4 configuration are shown in Fig 3. The left three disks are the data disks (DD 1, DD 2, DD 3 as written at the top of these disks) whereas the right most disk is the parity disk (PD as written at the top of this disk) in Fig 3

Write operation

Strips the data blocks (A, B, C, D, E, F, G, H, I) in the three data disks
Calculates parity block corresponding to each row of data blocks during each write operation and stores the parity block in the parity disk

Read operation

Reads the data blocks from the data disks and computes **new parity block** from these data blocks
Reads the parity block from the parity disk and compares this parity block with the **new parity block** to verify the data blocks for reliability

N+1 (N data disks + 1 parity disk, N=3 in Fig 3) is the number of disks

If a data disk fails, the data blocks of this faulty disk can be recovered by an appropriate calculation between the parity blocks and the data blocks of the remaining data disks.

Needs hardware support for parity calculation

Good for sequential read and write operation

Good for random read but bad for random write as parity block calculation causes large loss of time

Single disk error can be detected and corrected easily

Fig 4: RAID 5

DD 1	DD 2	DD 3	DD 4	DD 5
0	1	2	3	P0
4	5	6	P1	7
8	9	P2	10	11
12	P3	13	14	15
P4	16	17	18	19

Five disks (DD 1, DD 2, DD 3, DD 4, DD 5) of RAID 5 configuration are shown in Fig 4.

Write operation

Strips the data blocks (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19) in the five disks

Calculates parity block corresponding to each row of data blocks during each write operation

Distributes the parity blocks among disks

Read operation

Reads the data blocks from the disks and computes **new parity block** from these data blocks

Reads the parity block corresponding to the data blocks from the disk and compares this parity block with the **new parity block** to verify the data blocks for reliability

$N+1$ is the number of disks

If a data disk fails, the data blocks of this faulty disk can be recovered by an appropriate calculation between the parity blocks and the data blocks of the remaining data disks

Needs hardware support for parity calculation

Good for sequential read and write operation

Good for random read but bad for random write as parity block calculation causes large loss of time

Single disk error can be detected and corrected easily

Helps to perform multiple write operation simultaneously

[For example, write operation in data block 8 in DD 1 needs to update the parity block P2 in DD 3; again write operation in data block 5 in DD 2 needs to update the parity block P1 in DD 4. Four different disks (DD 1, DD 2, DD 3, DD 4) are required to access for performing these two write operations and hence can be done simultaneously.]

Fig 5: RAID 6

DD 1	DD 2	DD 3	DD 4
A	B	P1	P2
C	P3	P4	D
P5	P6	E	F

Four disks (DD 1, DD 2, DD 3, DD 4) of RAID 6 configuration are shown in Fig 5.

Write operation

Strips the data blocks (A, B, C, D, E, F) in the four disks

Calculates dual parity block corresponding to each row of data blocks during each write operation

Distributes the parity blocks among disks

Read operation

Reads the data blocks from the disks and computes **new parity blocks** from these data blocks

Reads the parity blocks corresponding to the data blocks from the disk and compares this parity blocks with the **new parity blocks** to verify the data blocks for reliability

N+2 is the number of disks

If a data disk fails, the data blocks of this faulty disk can be recovered by an appropriate calculation between the parity blocks and the data blocks of the remaining data disks

Needs hardware support for parity calculation

Good for sequential read and write operation

Good for random read but bad for random write as parity block calculation causes large loss of time

Can handle the failure of two disks

Complex to implement as it has to calculate two parity blocks for each write operation